Structure-Aware Cross-Dialect SQL Transfer for DBMS Fuzzing Seed Construction

***Abstract*—DBMS fuzzing mutates initial seed test cases to explore execution behaviors and trigger crash bugs, making the quality and diversity of the initial seeds critical. Built-in test suites are commonly used for this purpose, yet many DBMSs lack them. Consequently, test cases from mature systems are often reused, but this process is hindered by SQL dialect incompatibilities. Traditional SQL translation techniques primarily operate at statement-level and are not designed for complete test-case transfer for DBMS fuzzing. Recent fuzzing-oriented approaches instead employ LLMs for end-to-end test-case transfer. However, limited knowledge of SQL structures and dialects often introduces syntactic and semantic errors, while direct transfer may simplify SQL structures and cross-statement dependencies, reducing the diversity of generated seeds.**

To address these limitations, we propose MORPH to construct fuzzing seeds with structure-aware cross-dialect SQL transfer. Our key insight is to decouple structural information from SQL syntax via a dialect-independent Abstract Query Template (AQT), enabling test-case-level abstraction and dialect-aware instantiation. Specifically, MORPH first constructs a dialect-independent AQT from the source test case, preserving statement order and cross-statement dependencies. It then performs dialect-aware AQT instantiation to generate target-dialect SQL test cases using dialect-specific knowledge and constraints. Finally, MORPH leverages parser feedback from the downstream fuzzer to repair remaining incompatibilities, ensuring that the generated test cases are parsable and effectively mutable. We evaluate MORPH on four popular DBMSs: DuckDB, ClickHouse, MonetDB, and Virtuoso. Compared with the state-of-the-art seed transfer approach SEDAR, MORPH covers 14% more branches, finds 16 more bugs in 24 hours, and improves the overall execution validity of transferred seeds by 130%. More importantly, MORPH uncovers 28 previously unknown DBMS bugs, including 20 confirmed by developers and 14 already fixed.

***Index Terms*—DBMS Fuzzing, Initial Seeds, Cross-DBMS SQL Transfer**

I. INTRODUCTION

Database management systems (DBMSs) provide the essential framework for organizing and accessing data. Ensuring that a DBMS operates reliably is critical for maintaining data integrity and business continuity. However, the complex architectures and intricate implementations of modern DBMSs inevitably introduce potential bugs, which can lead to system crashes, data loss, or service interruptions [1], [2]. Timely detection of such bugs is therefore crucial for maintaining the reliability of these DBMSs.

In recent years, fuzzing has emerged as an effective technique for uncovering crashes and detecting hidden errors [3]–[6]. Many DBMS fuzzers have already discovered lots of bugs in popular DBMSs [7]–[9]. These fuzzers typically generate new test cases by repeatedly mutating a set of initial seed queries to explore more execution paths. Consequently, the effectiveness of fuzzing largely depends on the quality and diversity of the initial seeds [10]–[12]. While established DBMSs (e.g., PostgreSQL [13]) possess extensive test suites that serve as ideal seeds, emerging engines like DuckDB [14] often lack such assets, creating a critical bottleneck that limits the depth of testing for existing fuzzers.

To construct initial seeds for these DBMSs, a common practice is to reuse test cases from other mature systems, but SQL dialect incompatibilities often hinder this process. Existing SQL translation techniques [15], [16] primarily focus on statement-level translation. In contrast, DBMS fuzzing seeds are multi-statement test cases with rich cross-statement dependencies (e.g., schema references or type constraints). As a result, statement-level translation may break dependency chains, leading to execution failures when transferred across systems.

Recent fuzzing-oriented approaches instead leverage large language models (LLMs) for end-to-end test-case transfer [17]. For example, SEDAR [17] combines schema information and raw SQL text as input to an LLM to generate target-dialect seeds. However, such approaches still suffer from two fundamental limitations: (i) First, due to limited understanding of target SQL dialects and weak structural awareness of multi-statement programs, LLM-based translation may introduce syntactic or semantic errors during generation, leading to invalid or inconsistent query behaviors. (ii) Second, to satisfy strict grammar constraints imposed by downstream fuzzers, parser-incompatible fragments are handled after translation rather than resolved during it. For example, SEDAR temporarily comments them out during mutation and restores them before execution on the target DBMS, which retains the original query logic but leaves translation errors in these fragments unrepaired.

To address the limitations of existing approaches, we propose a structure-aware method for cross-dialect transfer of SQL test cases. Our key insight is to **decouple structural information from SQL syntax via a dialect-independent Abstract Query Template (AQT), enabling test-case-level abstraction and dialect-aware instantiation.** We observe that, although different SQL dialects vary in function naming or syntactic sugar, they typically share an *invariant abstract query logic* that governs the underlying operations and relationships between data elements. By separating the core query logic from dialect-specific syntax, test cases can be transferred across DBMSs while preserving structurally rich execution behaviors that are essential for fuzzing. This enables fuzzing to be more accurate and comprehensive, as it retains the essential behaviors of the test cases while avoiding transfer failures caused by superficial

syntactic differences.

However, realizing this idea has three challenges: (1) First, SQL logic is often tightly coupled with dialect-specific syntax, requiring the separation of syntactic differences while preserving the original structure, dependencies, and semantic richness. (2) Second, ensuring target-side execution validity during SQL transfer is non-trivial, as operators, functions, and type conversions may differ across dialects, requiring carefully constrained transfer. (3) Third, transferred test cases must satisfy strict parsing and mutation requirements imposed by downstream fuzzers, while preserving their structural richness under these constraints.

To address these challenges, we propose MORPH, a structure-aware cross-dialect DBMS fuzzing framework for constructing high-quality initial seeds and enabling effective fuzzing. MORPH consists of three steps: (1) First, it parses each SQL test case into an Abstract Query Template (AQT), capturing statement order, cross-statement dependencies, and clause-level structure while abstracting dialect-specific syntax. (2) Second, MORPH instantiates AQT nodes into target-dialect constructs under target-dialect constraints while propagating the evolving target schema context across statements, with localized rule generation and repair when necessary. LLMs are used for bounded rule generation or local repair under structural constraints. (3) Finally, it refines generated test cases to satisfy fuzzing requirements using runtime feedback, converting unsupported constructs into parsable forms while preserving their execution behavior as much as possible.

We implement MORPH and evaluate it on four popular DBMSs, namely DuckDB, ClickHouse, MonetDB, and Virtuoso. Although these DBMSs have been extensively tested, MORPH still uncovers 28 previously unknown bugs, including 20 confirmed and 14 already fixed. These bugs have received attention from developers. For example, all reported bugs of MonetDB have been incorporated into its official NEXTRELEASE milestone [18]. We further compare MORPH with state-of-the-art DBMS fuzzing tools, including SQLancer, SQLRight, SQUIRREL, GRIFFIN, SEDAR, and SmartFuzz. Across the four DBMSs under 24 hours, MORPH achieves 7%–528% higher branch coverage and discovers 5–22 more bugs than others. To evaluate seed transfer quality, we also compare MORPH against representative relevant techniques, including SQLGlot, jOOQ, CrackSQL, RISE, and SEDAR. MORPH outperforms these baselines by 5%–62% in syntax correctness, 100%–450% in execution validity, and 11%–253% in preservation of SQL functions. Finally, we conduct an ablation study to show the contribution of each component in MORPH. In summary, the paper makes the following contributions:

1) We find that many DBMSs lack high-quality initial seeds for efficient fuzzing. Transferring existing test cases requires coordinating cross-statement dependencies, target-side executability, and compatibility with downstream fuzzers.
2) We propose MORPH, a dependency-preserving whole-test-case seed transfer framework. Instead of translating isolated SQL statements, MORPH encodes each source test case as an AQT, instantiates it under target-dialect constraints, and adapts remaining incompatibilities into fuzzer-parsable forms while preserving testing-relevant structures.
3) We implement MORPH and evaluate it on four DBMSs, where it discovers 28 previously unknown bugs (20 confirmed, 14 fixed). Compared to the state-of-the-art method SEDAR, MORPH improves the overall execution validity of transferred seeds by 130%, achieving a 14% increase in branch coverage.

II. BACKGROUND AND MOTIVATION

SQL Dialects. *Database Management Systems (DBMSs)* underpin modern data-driven applications and expose high-level declarative query languages for data interaction. Although SQL is standardized, real-world DBMSs implement diverse *SQL dialects* with substantial syntactic and semantic variations. For example, PostgreSQL parses date strings with `TO_DATE` and the format `'YYYY-MM-DD'`, whereas DuckDB does not provide `TO_DATE` and instead uses `STRPTIME` with the format `'%Y-%m-%d'`. These dialect gaps pose substantial barriers for reusing test assets across different DBMSs.

DBMS Fuzzing. *DBMS Fuzzing* has emerged as a core technique for discovering database system crashes [9], [12], [19]. By continuously mutating existing SQL test cases to traverse diverse execution paths, DBMS fuzzers can effectively expose corner cases and potential bugs. The effectiveness of fuzzing largely depends on the quality and diversity of *initial seed test cases*. Due to the diverse features of DBMSs, fuzzers lacking high-quality initial seeds with specific syntactic features (e.g., recursive CTEs or window functions) are often confined to shallow parsing logic, leaving deep architectural bugs undetected. However, while mature DBMSs such as PostgreSQL provide rich test suites, many emerging systems lack such resources, creating a bottleneck for effective fuzzing.

Cross-Dialect Seed Transfer. To address seed scarcity for emerging DBMS testing, cross-DBMS SQL transfer has emerged as an active research direction [25], but this is challenging due to SQL dialect differences. As shown in Table I, existing tools provide limited support for preserving cross-statement dependencies during test-case-level transfer for fuzzing. Specifically, existing SQL translation techniques [15],

TABLE I
CAPABILITY COMPARISON OF EXISTING SQL TRANSLATION SYSTEMS FOR TEST-CASE-LEVEL DBMS FUZZING SEED TRANSFER.

Tool	DDL	DML	DQL	Test Case Migration	Cross-stmt. Deps.
SQLGlot [15], [20]	✓	✓	✓	△	×
jOOQ [16], [21]	✓	✓	✓	×	△
RISE [22]	△	△	✓	×	×
CrackSQL [23]	△	△	✓	×	×
QTRAN [24]	△	△	✓	×	×
SEDAR [17]	△	△	✓	✓	△
MORPH	✓	✓	✓	✓	✓

Cross-stmt. Deps. denotes explicit tracking of forward read-after-write dependencies and schema/type context across statements. ✓: supported and documented or evaluated; △: partially supported, i.e., supported only for some statements or settings, or through a separate component; ×: not supported.

[16] primarily focus on statement-level translation. In contrast, fuzzing seeds are multi-statement test cases that include complex interactions among DDL, DML, and DQL statements. SEDAR [17] transfers complete test cases, but it does not maintain translated target-side context across statements. MORPH instead represents test structures and forward read-after-write dependencies in an AQT, coordinates instantiation with evolving target-side schema and type context, and repairs seeds with parser and execution feedback before mutation.

Motivating Example. We next use an example to illustrate the limitations of LLM-based fuzzing-oriented seed transfer methods such as SEDAR. LLM-based cross-dialect transfer methods help in the generation of initial fuzzer seeds but may introduce *syntax or semantic errors* due to hallucinations. Moreover, many fuzzers cover only a subset of SQL grammar.

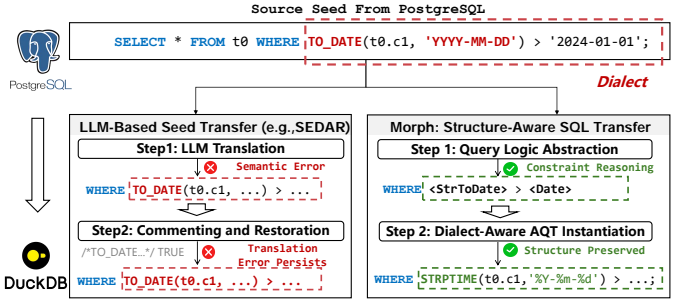


Fig. 1. Motivating example of running SEDAR on a PostgreSQL seed (Left): the incompatible fragment is commented out during mutation and restored before execution, leaving the translation error unrepaired. MORPH abstracts dialect-specific logic via AQT and instantiates it in the target dialect (Right).

Limitations of Existing Approaches. Figure 1 shows a test case transferred by SEDAR from PostgreSQL to DuckDB, which contains dialect-specific features such as `TO_DATE` and its format string. In this example, the LLM retains `TO_DATE` in the translated predicate. To satisfy the fuzzer's parser, SEDAR temporarily comments out this fragment and replaces the predicate with `TRUE` during mutation, then restores the fragment before execution. Since the restored fragment still contains `TO_DATE`, the mutated test case fails on DuckDB.

Basic Idea of MORPH. The core idea of MORPH is to thoroughly decouple logical intent from syntactic representation, shifting from unstable textual translation to more robust structural recursive substitution. As shown in Figure 1, MORPH abstracts the dialect-specific `TO_DATE` operation into an AQT node labeled `<StrToDate>`. For multi-statement test cases, it propagates target-side schema and type context in statement order and uses this context together with target-dialect information to instantiate AQT nodes. Parser and execution feedback guide the repair of remaining incompatibilities before mutation. In this example, MORPH instantiates `<StrToDate>` with `STRPTIME` and converts the format string, producing an executable seed that retains the date predicate.

III. APPROACH

MORPH is a structure-aware cross-dialect DBMS fuzzing framework for generating effective seeds for the target DBMSs. As Figure 2 shows, it first abstracts each source test case into an AQT that captures statement templates, cross-statement dependencies, and schema context, then recursively instantiates abstract nodes under target-dialect constraints using retrieved or generated rules. When generated outputs are rejected by the target parser or executor, MORPH performs feedback-driven local repair instead of discarding the structure, producing fuzzer-compatible seeds while preserving testing-relevant features.

A. Structure-Preserving Abstraction

This step decouples the logical intent of a test case from its dialect-specific syntax. The challenge is that dialect-specific constructs, such as proprietary functions, type casts, and interval operators, are often embedded within hierarchical query structures (e.g., nested subqueries and CTEs). As a result, extracting reusable logic requires preserving relational structure, scoping boundaries, and cross-statement dependencies, rather than relying on surface-level text replacement. We address this by formalizing each seed as an AQT that records ordered statement templates, dependency relations, and cumulative schema context as a structured object amenable to manipulation. We begin with the following definitions.

Formal Definition. To support syntax-aware query abstraction and decouple the logical intent of a test case, we introduce the SQL abstraction rules and AQT. They are defined as follows:

Definition 1 (SQL Abstraction Rule): An abstraction rule maps a dialect-specific SQL construct to a semantic label:

$$[\mathit{Function} \mid \mathit{Type} \mid \cdots] \rightarrow [\langle \mathit{FuncTag} \rangle \mid \langle \mathit{TypeCast} \rangle \mid \cdots].$$

where *Source* is a dialect-specific SQL element and *Target* is its corresponding abstract label that represents the semantic intent. For example, a rule for interval computation can be represented as $\texttt{AGE}(\cdot) \rightarrow \langle \text{TimeDiff} \rangle$.

Definition 2 (Abstract Query Template): Given a SQL test case $\mathcal{C}$, its AQT is a five-tuple:

$$\mathcal{T} = \langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle.$$

Here, $\mathbf{U} = \langle \mathcal{U}_1, \ldots, \mathcal{U}_n \rangle$ is the ordered sequence of statement templates, where each $\mathcal{U}_i = \langle kind_i, S^i_{norm}, \mathcal{G}^i, R_i, W_i \rangle$ records the statement type, normalized skeleton, statement-level intra-statement graph, read set, and write set of statement s_i. $\Delta \subseteq \{(i,j) \mid i < j\}$ captures cross-statement dependencies, where $(i,j) \in \Delta$ iff $R_j \cap W_i \neq \emptyset$. $\mathcal{D}$ records the cumulative schema context after executing $\langle s_1, \ldots, s_n \rangle$, including table definitions, column types, constraints, and object-state summaries that are propagated as source-side constraints during target-side instantiation. Each object-state summary additionally records whether a schema object exists and the schema changes induced by preceding DDL statements, and is updated incrementally in statement order. $\mathcal{G}_{tc} = (V_{tc}, E_{tc})$ is the test-case dependency graph, where $V_{tc} = \{1, \ldots, n\}$ and $E_{tc} = \Delta$; thus, Δ is the dependency set and $\mathcal{G}_{tc}$ is its graph representation, used to select dependency-relevant context during instantiation. $\mathcal{H}$ is a structural hash over the ordered normalized skeletons and dependency edges for deduplication and cache indexing.

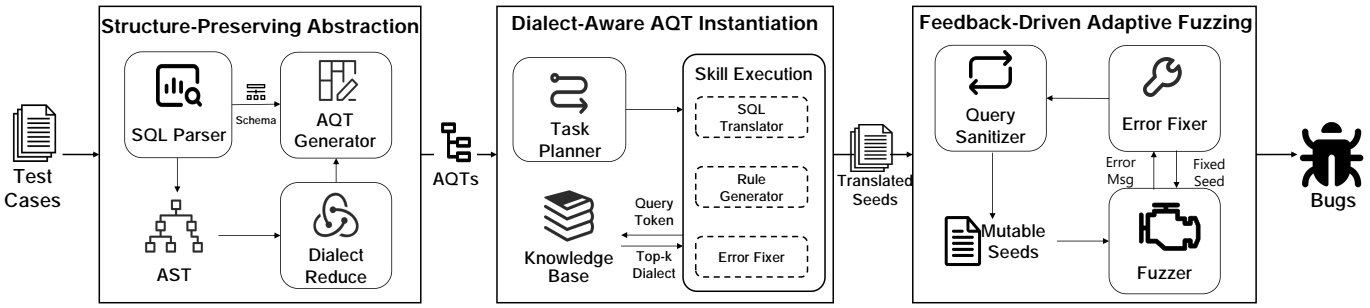


Fig. 2. **Overview of the MORPH.** MORPH generates high-quality cross-dialect test cases following three steps: (1) Structure-Preserving Abstraction. MORPH first parses SQL into an AST and extracts schema information, then replaces dialect-specific syntax with generic labels while preserving cross-statement dependencies to construct AQTs. (2) Dialect-Aware AQT Instantiation. MORPH uses a task planner to decompose test case transformations and instantiates AQT nodes by retrieving dialect evidence from a hybrid knowledge base, invoking the LLM only for bounded rule generation and local error correction. (3) Feedback-Driven Adaptive Fuzzing. It fixes syntax incompatibilities, then uses DBMS execution feedback to repair remaining errors, and finally marks mutable regions in the mutable seeds to guide fuzzer mutations.

Structure-Aware AQT Generation. Unlike prior statement-level translators that process each SQL statement in isolation and emit a statement-local rewrite rule, MORPH treats the whole test case as the abstraction unit. Algorithm 1 summarizes this extraction process. It first initializes the statement-unit list, dependency set, and extraction context. For each statement, EXTRACTSTMTTEMPLATE abstracts dialect-specific syntax into a normalized template while preserving scoped structures such as CTEs and subqueries (Line 3). MORPH then derives read/write sets, records cross-statement dependencies, appends the template, and updates the context for subsequent statements (Lines 4–7). After the loop, MORPH snapshots the final context, builds the dependency graph, computes the structural hash, and returns the AQT five-tuple (Lines 8–11). This representation enables SQL test cases to be manipulated as structured graph-based objects rather than flat strings.

Algorithm 1: AQT Extraction

Input : Test case $\mathcal{C} = \langle s_1, \ldots, s_n \rangle$; initial context $\mathcal{S}_0$; rules $\mathcal{R}$
Output : AQT $\mathcal{T} = \langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle$

1 $\mathbf{U} \leftarrow [\,]$; $\Delta \leftarrow \emptyset$; $Ctx \leftarrow \mathcal{S}_0$;
2 **foreach** s_i ***in*** $\mathcal{C}$ **do**
3 $\quad \mathcal{U}_i \leftarrow$ EXTRACTSTMTTEMPLATE$(s_i, Ctx, \mathcal{R})$;
4 $\quad (R_i, W_i) \leftarrow$ ACCESSSETS$(\mathcal{U}_i)$;
5 $\quad \Delta \leftarrow \Delta \cup \{(j, i) \mid j < i \wedge R_i \cap W_j \neq \emptyset\}$;
6 $\quad \mathbf{U}$.APPEND$(\mathcal{U}_i)$;
7 $\quad Ctx \leftarrow$ UPDATECONTEXT(Ctx, s_i) ; `// Evolve schema/state`
8 $\mathcal{D} \leftarrow Ctx$;
9 $\mathcal{G}_{tc} \leftarrow (\{1, \ldots, |\mathbf{U}|\}, \Delta)$;
10 $\mathcal{H} \leftarrow$ SHA256$(\bigoplus_i$ SKELETON$(\mathcal{U}_i) \oplus \Delta)$;
11 **return** $\langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle$;

For example, Figure 3 shows one statement from a source SQL test case containing a nested subquery, where the subquery filters records using an interval computation, e.g., "`SELECT ... WHERE u.created_at >(SELECT MIN(ts) FROM logs WHERE EXTRACT(YEAR FROM AGE(ts))::INT >= 1)`". The statement is first parsed into an AST, and structural boundaries are identified during recursive traversal. The `SUBQUERY` root node is recognized as a boundary, decomposing the query into an outer block (ID 1) and an inner subquery (ID 2). The abstraction engine then applies rules $\mathcal{R}$ to normalize dialect-specific constructs, mapping `AGE(...)`, `::INT`, and `MIN(.)` into <TimeDiff>, <TypeCast>, and <AggMin>. Finally, a statement template is constructed, capturing the parent-child relationship via a pointer <Ptr:2> and producing a normalized skeleton S^i_{norm} that preserves logic while abstracting dialect differences.

B. Dialect-Aware AQT Instantiation

This step instantiates AQTs into syntactically valid target seed sequences while preserving semantic-rich structures from the source seed. The dependency edges in $\mathcal{G}_{tc}$ identify dependency-relevant preceding statements whose target-side context is used during instantiation and repair, and the evolving target context is propagated across statements to maintain cross-statement type and reference consistency. The challenge is preserving target-side execution validity during recursive AQT-node instantiation without breaking the original test-case order. To address it, MORPH transforms the instantiation task into a closed-loop reasoning process built on a Retrieval-Augmented

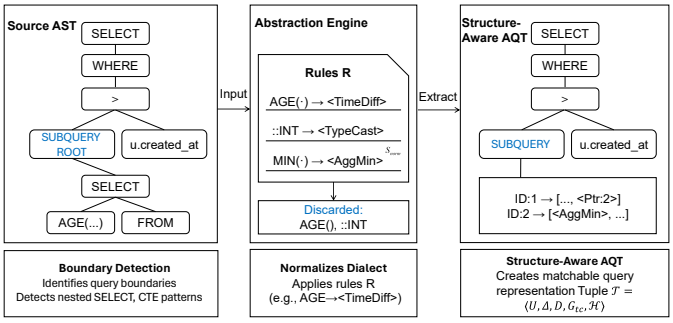


Fig. 3. An example of Structure-Preserving Abstraction. It first detects and normalizes dialect-specific SQL features to abstract labels using abstraction rules, and produces structure-aware AQTs.

Generation (RAG) architecture, which grounds LLM outputs in retrieved dialect-specific evidence to reduce hallucinations.

We formulate dialect translation as a two-level instantiation problem in which the outer level processes statements in their original order and uses the statement dependency graph to select the context for each statement, and the inner level recursively instantiates each statement-level graph $\mathcal{G}^i$ in a bottom-up manner. To keep the generated queries valid in the target dialect

Algorithm 2: AQT Instantiation

Input : AQT $\mathcal{T} = \langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle$; target dialect D_t; Hybrid KB HKB; threshold δ
Output : Compatible seed sequence $\mathcal{Q}_{target}$

1 $\mathcal{Q}_{target} \leftarrow [\,]$; $C_s \leftarrow \mathcal{D}$; $C_t \leftarrow \emptyset$;
2 **for** $i \leftarrow 1$ **to** $|\mathbf{U}|$ **do**
3 $\quad C_t^{(i)} \leftarrow \textsc{SelectContext}(C_t, \mathcal{Q}_{target}, \Delta, i)$; `// Dependency-relevant context`
4 $\quad Q_i \leftarrow \textsc{InstantiateStmt}(\mathcal{U}_i, D_t, HKB, \delta, C_s, C_t^{(i)})$;
5 $\quad \mathcal{Q}_{target}.\textsc{Append}(Q_i)$;
6 $\quad C_t \leftarrow \textsc{UpdateContext}(C_t, Q_i)$; `// Propagate context`
7 $\textsc{IndexArtifact}((\mathcal{H}, D_t), \mathcal{Q}_{target})$; `// Cache key outside recursion`
8 **return** $\mathcal{Q}_{target}$;

during test-case instantiation, the system employs a Dialect-Aware AQT Instantiation process as formalized in Algorithm 2. At the *outer level*, the algorithm processes statements in their original order. For each statement s_i, SELECTCONTEXT uses the dependency set Δ to select its direct predecessors $\{j \mid (j,i) \in \Delta\}$ (Line 3). Among the successfully instantiated predecessors, it keeps only the most recent one for each set of objects shared with s_i, since earlier writes to the same objects are superseded, and retains at most five predecessors. It returns their source and translated statements together with the target-side object-state summaries of the objects read by s_i. It then invokes INSTANTIATESTMT with the source constraints and the selected target context, appends the generated query, and updates the target context for subsequent statements (Lines 4–6). After all statements are instantiated, the generated target sequence is indexed by $(\mathcal{H}, D_t)$ for future reuse. At the *inner level*, INSTANTIATESTMT recursively instantiates the statement-level graph using retrieved or generated local mappings and target-parser feedback to keep fragments executable. Finally, the instantiated statements form the target seed sequence $\mathcal{Q}_{target}$.

Test-Oriented Skill Execution. The above instantiation procedure is realized through *Test-Oriented Skill Execution*, a method that coordinates three specialized skills under the target-dialect constraints.

SQL Translator: This skill instantiates a token into a dialect-compliant fragment by integrating the input segment with function signatures from the *Vector Index*. For example, it maps `<TimeDiff>` to DuckDB's `date_sub` signature.

Rule Generator: When retrieval confidence for a feature falls below a threshold, this skill synthesizes a candidate local rule via the LLM's zero-shot reasoning over the AST context and few-shot examples. The candidate is cached in the Hybrid KB only after it passes target parser validation.

Error Fixer: When the target parser rejects a fragment, this skill localizes the error using both the compiler feedback and the AST context, then applies structure-preserving rewrites via constrained few-shot examples. This avoids degenerate repairs such as clause removal, keeping the output a *Compatible Seed* with its original complexity.

The three skills are grounded in a Hybrid KB with two complementary stores: the *Abstract Pattern Store* keeps deduplicated structural skeletons (e.g., `JOIN`/`WINDOW` templates) to preserve query topology, while the *Vector Index* retrieves top-k dialect evidence (function signatures and operator semantics) for each token τ, grounding generation in concrete evidence rather than free-form synthesis.

As shown in Figure 4, since `ID1` depends on the nested segment `ID2` (*Order*=[`ID2` → `ID1`]), MORPH first instantiates subquery `ID2` by mapping `<AggMin>` to `MIN(..)`, `<TimeDiff>` to DuckDB's `date_sub`, and `<TypeCast>` to a dialect-valid `CAST(.. AS ..)`, and validates it via parser feedback. It then substitutes the validated fragment into the outer predicate of `ID1`, yielding a *Compatible Seed* that preserves the nested time-window logic.

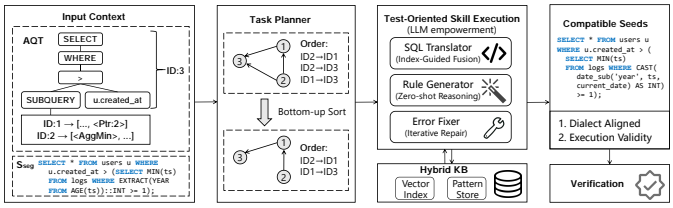


Fig. 4. An example of the RAG-enhanced Dialect-Aware AQT Instantiation.

C. Feedback-Driven Adaptive Fuzzing

Some DBMS features exceed the grammar supported by existing fuzzers, making otherwise executable target seeds difficult to mutate. The challenge is to adapt such features into fuzzer-compatible forms while preserving syntax correctness, semantic consistency, and mutation opportunities. MORPH addresses this challenge using parsing and execution feedback to repair unsupported constructs through a structure-preserving fallback process. As shown in Figure 5, compatible seeds are sanitized, executed in the target DBMS, repaired when necessary, and annotated with mutable regions for fuzzing.

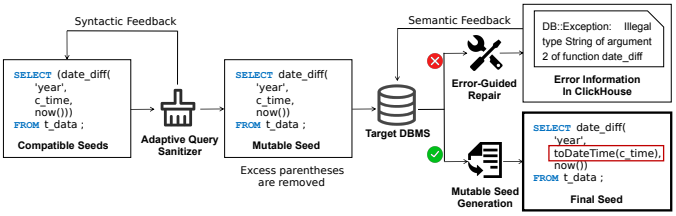


Fig. 5. Collaborative Workflow of Adaptive Query Sanitizing, Error-Guided Repairing, and Mutable Seed Generation in Feedback-Driven Adaptive Fuzzing.

Specifically, when a transferred seed Q_{target} still violates the fuzzer grammar, the sanitizer localizes the unsupported construct to its AQT node and repairs it using parser feedback rather than discarding it. For example, it removes excess brackets around `date_diff` in Figure 5, producing a syntactically valid Mutable Seed.

When an error signal is received (e.g., "Syntax error near SELECT"), MORPH applies four fallbacks in order: existing sanitization mappings, RAG-retrieved replacements, source-side execution followed by constant substitution, and finally minimal commenting. Only if the first three stages fail does MORPH comment out the minimal offending fragment. Constant substitution and minimal commenting may change the behavior of transferred seeds. Once a seed passes the fuzzer parser, MORPH marks data-dependent literals and predicates as mutable regions. This guides mutation toward logic-altering changes while keeping the repaired syntactic skeleton stable.

IV. EVALUATION

MORPH is implemented in Python and integrates GRIFFIN [10], a coverage-guided mutation fuzzer, as the downstream execution engine. For the LLM used in Test-Oriented Skill Execution and rule generation, we employ Gemini 2.5-Flash-Lite [26] as the primary language model due to its strong code understanding capabilities and cost efficiency. We set the temperature parameter to 0.2 to balance determinism and output diversity, as justified by the sensitivity analysis discussion in Section V. We evaluate MORPH to assess its performance in cross-dialect seed transformation and its practical impact on DBMS fuzzing effectiveness. Specifically, our evaluation focuses on the following research questions:

- **RQ1:** Can MORPH find unknown bugs in DBMSs?
- **RQ2:** How does MORPH compare to other fuzzing tools?
- **RQ3:** How does MORPH compare to other translation methods in terms of seed validity and semantic richness?
- **RQ4:** What is the contribution of each component in MORPH to the overall performance?

A. Experimental Setup

Target DBMSs and Environment. To evaluate the generalization of MORPH across diverse architectures, we selected four widely used DBMSs according to the DB-Engines Ranking [27]: MonetDB [28] v11.51, Virtuoso [29] v7.2, DuckDB [14] v1.4.3, and ClickHouse [30] v24.8 LTS. These systems cover diverse OLAP and multi-model architectures with distinct SQL dialects. We did not include traditional row-store OLTP systems, such as MySQL and PostgreSQL, as target DBMSs because they are used as source DBMSs for seed collection. Instead, we prioritized OLAP and multi-model systems to more rigorously evaluate MORPH's ability to bridge substantial dialect and architectural gaps, such as transferring seeds from row-store systems like MySQL to column-store systems like DuckDB. All experiments were conducted on a Linux server equipped with an Intel Xeon CPU and 256 GB of RAM. To ensure reproducibility and isolation, each DBMS instance ran within a separate Docker container.

Collected Test Cases. Following the prior methodology [17], we collected built-in test cases from mature DBMS repositories (i.e., SQLite [31], MySQL [32], and PostgreSQL [33]) for transfer. The dataset aggregates 447,357 SQL statements across 6,875 test files, totaling 55.7 MiB of query text. For each target DBMS, all methods use the same number of initial seeds, i.e., 5,879, 4,551, 5,850, and 5,990 seeds for MonetDB, Virtuoso, DuckDB, and ClickHouse, respectively.

Evaluation Metrics. We employ three metrics to evaluate MORPH. First, to assess transfer quality, we measure *Syntactic Correctness*, defined as the proportion of generated seeds that pass the target parser, and *Execution Validity*, defined as the proportion of generated seeds that execute without runtime errors. Here, execution validity is defined by executability in the target DBMS without runtime errors, not by result equivalence between the source and target DBMSs. We separately measure semantic richness by counting valid functions, data types, and keywords preserved in executable transferred seeds. Second, to quantify the effectiveness in improving fuzzing, we use LLVM's SanitizerCoverage to track *Branch Coverage* over 24-hour campaigns. We report the average results across five independent runs to mitigate randomness, and use the Mann–Whitney U test to assess statistical significance. Third, for real-world bug discovery, we report the number of detected unique bugs.

B. RQ1: Real-World Bug Detection

RQ1 evaluates MORPH's real-world bug-finding impact over the full campaign. We deployed MORPH on the development versions of the four target DBMSs for seven days each. As summarized in Table II, the campaign uncovered 28 unique bugs. Among these issues, 20 have been confirmed by the vendors, of which 14 critical bugs have already been fixed.

These bugs have received positive feedback and careful attention from the corresponding database developers. For instance, regarding the fixed bug in DuckDB (#205xx), the developers appreciated the report exposing an intricate type inference flaw in the *Executor*, which was triggered by

TABLE II
STATISTICS OF NEW DBMS BUGS DETECTED BY MORPH.

Confirmed/Detected	Component	Bug Type
MonetDB (15/17)	optimizer	SEGV(6), NPD(1), AF(1)
	server	SEGV(4), DBZ(1), AF(2)
	function	SEGV(1), NPD(1)
Virtuoso (2/5)	server	SEGV(4), UAF(1)
DuckDB (2/4)	executor	AF(1), DBZ(1)
	binder	SEGV(1)
	function	OOM(1)
ClickHouse (1/2)	function	SEGV(1)
	server	AF(1)
Total	9 components	**20 confirmed, 28 detected**

UAF: Use-After-Free; **AF**: Assertion Failure; **NPD**: Null Pointer Dereference; **DBZ**: Divide-by-Zero; **SEGV**: Segfault; **OOM**: Out of Memory.

preserving complex cross-table correlations from PostgreSQL. They merged a fix within days, further validating that MORPH successfully identified deep-seated bugs missed by existing test suites. Moreover, all reported bugs of MonetDB have been incorporated into its official NEXTRELEASE milestone [18]. The detected bugs are not limited to the surface level (e.g., Parser) but penetrate deep into the DBMS kernels. As shown in the second column of Table II, the bugs span critical subsystems including the Optimizer, Binder, and Storage Engine.

Case Study: An internal error in DuckDB. To illustrate the effectiveness of MORPH, we examine a fixed bug in DuckDB (#205xx) triggered by a complex query involving a nested `CROSS JOIN` and specific DDL configurations. Generally, in SQL dialects, join operations and subqueries are standard; however, the structural interaction between specific join types and aliased subqueries can induce a crash bug in DuckDB when executing the statement shown in the PoC. In this case, the source logic was derived from a PostgreSQL test case featuring a `CREATE TABLE` statement with `DOUBLE PRECISION` and a query containing a nested `CROSS JOIN`.

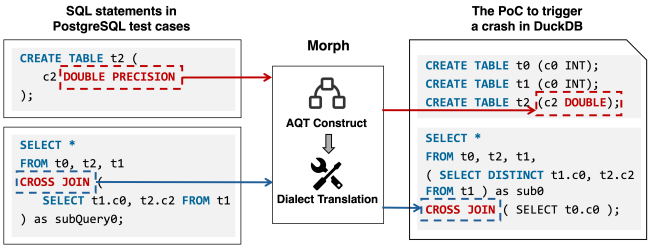


Fig. 6. An internal error in DuckDB (Bug #205xx). MORPH translates a test case of PostgreSQL to DuckDB while preserving data types and join logic.

MORPH's effectiveness stems from its structure-aware transfer. Take the case as an example, in PostgreSQL, nested joins involving a table with `DOUBLE PRECISION` and a subquery are well tested, whereas DuckDB lacks sufficient coverage for such combinations. Conventional approaches often simplify schemas (e.g., converting `DOUBLE` to `INT`) or flatten nested queries to ensure compatibility, sacrificing semantic fidelity. In contrast, MORPH preserves the full structural complexity of the original seed, forcing DuckDB to perform precise type inference and vector allocation for correlated joins and thereby exercising execution paths that simplified queries would not reach. This high-fidelity transfer exposes a critical flaw in the executor logic that remains undetected when structural complexity is removed.

Answer to RQ1: MORPH successfully exposed 28 unknown bugs in four popular DBMSs, affecting core components including the Optimizer and Storage Engine. Among them, 20 bugs have been confirmed, and 14 bugs have already been fixed by DBMS vendors.

C. RQ2: Comparison with Other Fuzzing Tools

RQ2 evaluates MORPH's effectiveness by comparing it to other fuzzing tools under the same setting. We compared MORPH against six state-of-the-art DBMS testing tools in 24 hours: SQLancer [34] (logic/crash baseline [10]–[12]), SQLRight [35] (coverage-guided logic-bug fuzzer), SQUIRREL [12] (default built-in seed corpus [36]), GRIFFIN [10] (grammar-free mutation baseline), SEDAR [17], and SmartFuzz [37] (LLM-based seed generation). For SQLancer, SQUIRREL, GRIFFIN, and SmartFuzz, we use the default configurations, seed corpora, and command lines from their official repositories, which are included in our artifact. We adapt SQLRight to the four target DBMSs, and the adaptation code is also included in our artifact. After fuzzing, we normalized coverage via a unified dry run across all configurations, and deduplicated crashes using stack-trace hashing followed by manual inspection for verification.

As illustrated in Figure 7, the transferred seeds lead to the highest total branch coverage, covering 422,130 branches across the four DBMSs. This corresponds to overall improvements of 528%, 518%, 60%, 54%, 14%, and 7% over SQLancer, SQLRight, SQUIRREL, GRIFFIN, SEDAR, and SmartFuzz, respectively, on their supported targets. Figure 7 further shows continued branch gains after 12 hours, while several baselines plateau sooner. On Virtuoso, MORPH grows more slowly than SmartFuzz at first due to the larger dialect gap, but reaches comparable coverage.

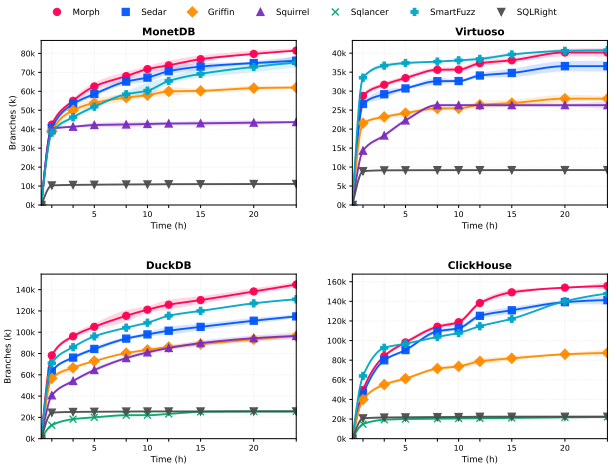


Fig. 7. Coverage growth over 24-hour fuzzing campaigns. Displayed are the means and the 95% confidence intervals of five runs.

TABLE III
NUMBER OF BUGS DETECTED BY DIFFERENT TOOLS (24H).

DBMS	SQLancer	SQLRight	SQUIRREL	GRIFFIN	SEDAR	SmartFuzz	MORPH
MonetDB	-	0	0	0	3	11	**14**
Virtuoso	-	0	0	0	2	**3**	**3**
DuckDB	0	0	0	0	1	2	**3**
ClickHouse	0	0	-	0	0	1	**2**
Total	0	0	0	0	6	17	22
Increment	22↑	22↑	22↑	22↑	16↑	5↑	-

The coverage improvement also brings more detected bugs. As shown in Table III, the campaign exposes significantly more unique bugs than the baselines, including 16 more bugs than SEDAR and 5 more bugs than SmartFuzz across the four DBMSs. Bug counts are the union of unique bugs across the five runs after stack-trace deduplication and manual verification. We further pair existing translators with the same downstream fuzzer on DuckDB: MORPH covers 33.6% and 16.7% more branches than SQLGlot+GRIFFIN and RISE+GRIFFIN, respectively, and detects 3 more bugs than each. To control for seed quantity, we also randomly sample 100, 200, and 300 seeds generated by MORPH and SEDAR as the initial seeds of GRIFFIN. Under the same seed count, MORPH achieves 17.2%–92.4% higher coverage than SEDAR across the four DBMSs, which indicates that the improvement does not stem from a larger seed corpus.

Complementarity with SmartFuzz. SmartFuzz and MORPH construct seeds from different sources (Section VI). We further run GRIFFIN with the union of their seeds, which covers 413,021 branches and detects 35 bugs in total, i.e., 13 and 18 more bugs than MORPH and SmartFuzz alone, respectively. However, its coverage is lower than that of MORPH alone on MonetDB and DuckDB, where the additional seeds reduce the execution throughput, leaving fewer mutation opportunities for potentially more valuable seeds.

These results demonstrate the practical utility of the transferred seeds for downstream fuzzing, yielding higher branch coverage and more detected bugs in the evaluated campaigns. To examine one source of this difference, we next compare the mutation-handling strategies under controlled conditions.

Controlled Comparison of Mutation Handling. MORPH repairs incompatibilities before mutation, whereas SEDAR uses comment-and-restore handling (Section II). Neither strategy guarantees semantic equivalence of the mutated queries. For each target DBMS, we randomly sampled 300 translated seeds and applied both strategies to the same seeds. Each configuration was run with GRIFFIN for 24 hours. The translated inputs, target database initial state, GRIFFIN version and configuration, and resource allocation were held fixed; only the mutation-handling strategy differed. Across the four DBMSs, MORPH's handling achieves 27.2%, 15.5%, 4.0%, and 9.4% higher branch coverage than SEDAR's handling on DuckDB, MonetDB, Virtuoso, and ClickHouse, respectively ($p < 0.05$). These results show the coverage benefit of MORPH's handling strategy under the evaluated conditions.

Answer to RQ2: MORPH covers 528%, 518%, 60%, 54%, 14%, and 7% more branches than SQLancer, SQLRight, SQUIRREL, GRIFFIN, SEDAR, and SmartFuzz, respectively, and discovers 22, 22, 22, 22, 16, and 5 more bugs, respectively. The results demonstrate the effectiveness of MORPH in enhancing DBMS fuzzing.

D. RQ3: Effectiveness of Transfer of MORPH

For translation effectiveness, we compare MORPH against six baselines, including No translation, SQLGlot [15], jOOQ [16], CrackSQL [23], RISE [22], and SEDAR [17]. The No translation strategy directly executes the original test cases collected from source DBMSs without modification. SQLGlot and jOOQ represent off-the-shelf SQL dialect translators, while CrackSQL and RISE represent recent SQL translation systems. SEDAR represents the current state-of-the-art LLM-based transfer tool in constructing initial seeds in fuzzing. We used the original SEDAR binary obtained from the authors of SEDAR [17], which runs its complete pipeline (Section II). Our artifact includes the SEDAR binary, its configuration, and the SEDAR-generated seeds used in our experiments. Each baseline follows its original translation workflow, using the official SQL documentation of each target DBMS as dialect knowledge when required. CrackSQL and RISE translate each statement individually without schema context, and all methods use the same source test cases without additional manually written translation rules.

We evaluate these methods in terms of transferred-seed validity, SQL-feature richness, and source-to-target equivalence on sampled test cases. Validity and richness reflect fuzzing seed quality: valid seeds must execute on the target DBMS, and rich seeds should retain diverse SQL features for mutation. Equivalence measures whether transferred seeds preserve the query results and database states of the source test cases.

Validity of the Transferred Test Cases. As shown in Table IV, MORPH outperforms the baseline methods by 5%–62% in syntax correctness and 100%–450% in execution validity. Directly executing source seeds yields low execution validity, dropping to 0.097 on Virtuoso and reaching only 0.284 even in the best case. Existing translators improve syntactic acceptance but still leave an execution-validity gap; on DuckDB, SQLGlot, CrackSQL, and RISE reach at least 0.738 syntactic correctness, yet no more than 0.314 execution validity.

Existing SQL translation tools are designed for query migration rather than fuzzing seed transfer. For example, RISE processes one self-contained query at a time and does not maintain context across the statements of a test case. In the SQLite-to-DuckDB setting, our test-case-level transfer improves execution validity from 0.314 to 0.748. When the translated outputs are used as initial seeds for fuzzing, MORPH further achieves 16.7% higher branch coverage than RISE and detects three bugs in DuckDB, whereas RISE triggers none.

Compared with SEDAR, MORPH performs better in each DBMS, with 5%–19% higher syntactic correctness and 100%–272% higher execution validity across the tested DBMSs. As discussed in Section II (Table I), these low execution-validity scores stem from limited support for explicit cross-statement dependency preservation in translators that process one statement or query at a time, such as SQLGlot, jOOQ, CrackSQL, and RISE, or from the absence of translated target-side context across statements in SEDAR, which can break type or reference consistency when target-side schema changes are not propagated. These gains confirm that Dialect-Aware AQT Instantiation maps source intent into target-dialect constraints while preserving parsability.

Source-to-Target Result and State Equivalence. We randomly sampled 100 MySQL test cases and used the same

TABLE IV
SYNTACTIC CORRECTNESS AND EXECUTION VALIDITY OF TRANSFERRED SEEDS.

Method	MonetDB		Virtuoso		DuckDB		ClickHouse	
	Syn.	Exec.	Syn.	Exec.	Syn.	Exec.	Syn.	Exec.
No translation	0.589	0.236	0.324	0.097	0.674	0.284	0.613	0.182
SQLGlot	-	-	-	-	0.738	0.186	0.665	0.193
jOOQ	-	-	-	-	0.652	0.136	0.554	0.175
CrackSQL	-	-	-	-	0.740	0.211	0.717	0.264
RISE	0.830	0.266	0.545	0.178	0.857	0.314	0.764	0.278
SEDAR	0.781	0.388	0.562	0.169	0.895	0.365	0.821	0.315
MORPH	**0.929**	**0.776**	**0.670**	**0.629**	**0.939**	**0.748**	**0.895**	**0.694**
vs. SEDAR	+19%	+100%	+19%	+272%	+5%	+105%	+9%	+120%

Syn./Exec.: syntactic correctness/execution validity; "-": unsupported dialects.

cases to evaluate SQLGlot, RISE, SEDAR, and MORPH for transfer to DuckDB. We compared query results between the source and target executions for result equivalence, and compared the resulting database states, including schemas and table contents, for state equivalence. MORPH achieves 52.0% result equivalence and 74.0% state equivalence, compared with 14.0%/56.0% for SQLGlot, 34.0%/65.0% for RISE, and 38.0%/69.0% for SEDAR.

Semantic Richness of the Transferred Test Cases. Beyond validity, we measure whether transferred seeds retain testing value by counting valid functions, data types, and keywords. As shown in Table V, MORPH preserves on average 126% more functions, 60% more data types, and 73% more keywords than baselines. These counts show that MORPH produces transferred seeds with richer SQL features. Its dialect-guided AQT instantiation and feedback repair aim to adapt complex structures to the target DBMS and downstream fuzzer.

TABLE V
VALID SQL FEATURES IN TRANSFERRED SEEDS.

Method	MonetDB			Virtuoso			DuckDB			ClickHouse		
	F	DT	K	F	DT	K	F	DT	K	F	DT	K
No translation	107	29	235	28	16	102	185	34	291	83	36	124
SQLGlot	-	-	-	-	-	-	195	28	288	103	35	142
jOOQ	-	-	-	-	-	-	187	22	253	94	34	104
CrackSQL	-	-	-	-	-	-	198	32	329	146	46	136
RISE	145	27	236	45	18	122	251	34	314	155	54	160
SEDAR	132	29	244	52	17	136	214	39	340	227	61	152
MORPH	**161**	**29**	**370**	**87**	**24**	**335**	**456**	**63**	**476**	**332**	**64**	**187**
vs. SEDAR (%)	+22	+0	+52	+67	+41	+146	+113	+62	+40	+46	+5	+23

F/DT/K: functions/data types/keywords; "-": unsupported dialects.

Answer to RQ3: MORPH outperforms baselines by 5%–62% in syntax correctness and 100%–450% in execution validity. It also preserves on average 126% more functions, 60% more data types, and 73% more keywords than baselines. The results demonstrate that the transfer methods in MORPH produce seeds that are both executable and rich in testing-relevant logic.

TABLE VI
DEPENDENCY-GRAPH ABLATION ACROSS ALL SOURCE-TARGET PAIRS.

Variant	Exec.	Preds	Rel. (%)	Tokens
MORPH$_{full}$	**0.716**	0.47	100.0	509.6
MORPH$_{win}$	0.599	1.19	22.6	578.4
MORPH$_{all}$	0.643	27.95	4.2	1,842.3
MORPH$_{w/o\text{-}ctx}$	0.523	0.49	100.0	488.5

Preds: predecessors per statement; Rel.: dependency-relevant predecessors; Tokens: input tokens per statement.

E. RQ4: Contribution of Individual Components

To analyze the specific contributions of the individual components within MORPH, we conducted an ablation study across two phases. For the *translation phase*, we evaluated four main variants on all source-target pairs used in RQ3: (1) MORPH$_{base}$: the baseline using direct zero-shot LLM translation (i.e., prompting the LLM with the raw SQL and asking for a direct translation, unlike the No translation baseline which executes the raw, untranslated SQL directly); (2) MORPH$_{abs}$: equipped solely with *Structure-Preserving Abstraction*; (3) MORPH$_{rag}$: equipped solely with *Dialect-Aware AQT Instantiation*; (4) MORPH$_{full}$: the complete framework integrating both components. To further isolate cross-statement context, we compare MORPH$_{full}$ with three context variants: (5) MORPH$_{w/o\text{-}ctx}$: selects the same predecessors but provides only their source-side statements, without translated statements or target-side object-state summaries; (6) MORPH$_{win}$: replaces dependency-based context selection with the five most recent statements and the same object summaries; (7) MORPH$_{all}$: provides all preceding statements and the complete context.

For the *fuzzing phase*, since the *Feedback-Driven Adaptive Sanitizer* operates independently during runtime fuzzing and does not affect translation quality, we further introduce MORPH$_{direct}$ as a controlled variant where the runtime feedback loop is disabled and invalid seeds are directly discarded, and compare it against MORPH$_{full}$ where the sanitizer actively repairs parsing errors to maximize seed acceptance.

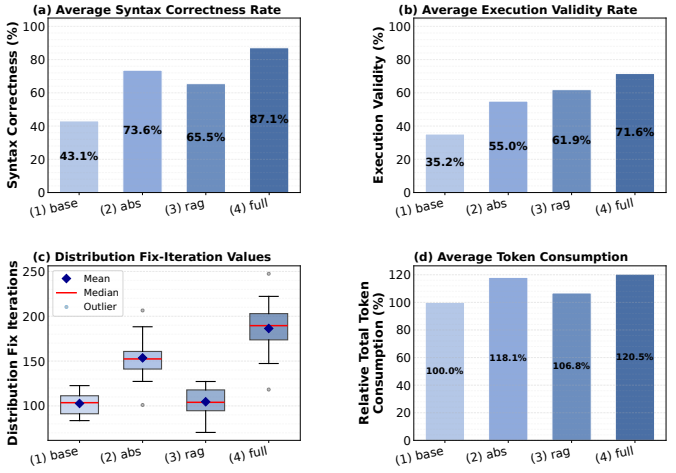


Fig. 8. Ablation Analysis of MORPH Components. Comparison of syntactic correctness/execution validity and resource consumption across four variants.

Contribution of Structure-Preserving Abstraction and Dialect-Aware AQT Instantiation. As shown in Figure 8 (a) and (b), when both components are disabled, MORPH$_{base}$ achieves 43.1% syntactic correctness and 35.2% execution validity. *Structure-Preserving Abstraction* mainly improves syntactic correctness: MORPH$_{abs}$ raises it to 73.6% (+30.5%) by providing a stable AQT skeleton, and raises execution validity to 55.0%. *Dialect-Aware AQT Instantiation* mainly improves execution validity: MORPH$_{rag}$ raises it to 61.9% (+26.7%) by grounding generation in retrieved dialect evidence, and raises syntactic correctness to 65.5%. With both components, MORPH$_{full}$ achieves the highest syntactic correctness (**87.1%**) and execution validity (**71.6%**).

Beyond dialect grounding, *Dialect-Aware AQT Instantiation* also propagates schema and type context across statements to preserve test-case-level dependencies. To isolate this effect, we compare MORPH$_{full}$ with MORPH$_{w/o\text{-}ctx}$. With nearly the same predecessors, execution validity drops from 0.716 to 0.523 (Table VI). Thus, context propagation mainly improves test-case executability. Figure 8(c) and (d) show that MORPH$_{full}$ incurs more repair iterations and token overhead, but delivers the best trade-off between quality and efficiency.

Contribution of the Dependency Graph. To isolate the dependency graph from context propagation, we compare MORPH$_{full}$ with MORPH$_{win}$ and MORPH$_{all}$. As shown in Table VI, MORPH$_{full}$ achieves the highest execution validity (0.716 vs. 0.599 and 0.643) with 11.9% and 72.3% fewer input tokens per statement, whereas only 22.6% and 4.2% of the predecessors provided by MORPH$_{win}$ and MORPH$_{all}$ are dependency-relevant. Thus, the dependency graph provides a compact, dependency-relevant context.

Contribution of Feedback-Driven Adaptive Sanitizer. To evaluate the contribution of the component, we compared the full framework (MORPH$_{full}$) against a variant named MORPH$_{direct}$, where the runtime feedback loop is disabled. In the static variant, generated seeds are fed directly to the fuzzer without adaptive repair; if they fail parsing, they are simply discarded. Table VII presents MORPH covers 8% to 39% more branches across four DBMSs than MORPH$_{direct}$ over a 24-hour campaign ($p < 0.05$). When the component is disabled, MORPH$_{direct}$ experiences a high rejection rate by the fuzzer at the parser stage due to syntax mismatches. In contrast, by actively repairing these specific errors, MORPH$_{full}$ recovers these invalid seeds, allowing them to pass the frontend and be used for fuzzers to explore more state space of target DBMSs.

Answer to RQ4: All components contribute to MORPH. Structure-Preserving Abstraction and Dialect-Aware AQT Instantiation improve syntactic correctness by 30.5% and execution validity by 26.7%, respectively, and together by 44.0% and 36.4%. The dependency graph achieves the highest execution validity with 72.3% fewer input tokens than the full context. The *Feedback-Driven Sanitizer* increases branch coverage by up to 39%.

TABLE VII
SANITIZER ABLATION ON BRANCH COVERAGE (24H).

DBMS	MORPH$_{direct}$	MORPH$_{full}$	Improvement
MonetDB	67,620	**81,486**	**+21%**
Virtuoso	37,213	**40,183**	**+8%**
DuckDB	118,453	**144,822**	**+22%**
ClickHouse	112,083	**155,639**	**+39%**

V. DISCUSSION

LLM Parameter Selection. To ensure reproducibility, we conduct a sensitivity analysis on the temperature parameter (0.2, 0.5, 0.8). The results show that $T = 0.2$ achieves a 95% translation success rate and 91% first-attempt stability, outperforming $T = 0.5$ (92%/78%) and $T = 0.8$ (85%/70%). Based on these results, we set $T = 0.2$ for all experiments to ensure both reliability and reproducibility.

Impact of the Underlying LLM. We justify the selection of Gemini 2.5-Flash-Lite for MORPH using experiments on MonetDB with different LLMs. Although Qwen3.5-27B achieves slightly higher execution validity (0.852), it incurs 39.7× higher cost and 9.5× longer migration time. Gemini provides a better balance of accuracy (0.776), cost, and throughput, making it suitable for large-scale seed generation. Processing all test files (65.1 statements per test suite on average) with Gemini 2.5-Flash-Lite costs $96 in total across the four target DBMSs, i.e., $0.004 and 219 seconds per seed, compared with $0.006 and 146 seconds per seed and a total cost of $144 for SEDAR. Under the same GPT-3.5-Turbo setting as SEDAR, MORPH still outperforms it in syntactic correctness (0.89 vs. 0.781) and execution validity (0.533 vs. 0.388), indicating that the gains mainly come from the proposed method rather than model capacity.

Extensibility and Manual Effort. To extend MORPH to another DBMS, we first automatically extract lexical and syntactic features from grammar specifications and register the corresponding mapping rules into the Hybrid KB. For features that cannot be mapped automatically, MORPH uses the LLM to generate candidate rules, followed by manual review and correction. To validate the generated rules, we apply them to translated test cases and check the resulting translations for errors; rules that produce incorrect translations are manually inspected and corrected. Table IX summarizes the extension effort across four target DBMSs. Overall, only 4.2% (136/3,212) of transfer rules require manual correction,

TABLE VIII
LLM COMPARISON. SYN.: SYNTACTIC CORRECTNESS; EXEC.: EXECUTION VALIDITY; TS: TEST SUITE.

Model	Syn.	Exec.	Cost/TS ($)	Time/TS (s)
Gemini 2.5-Flash-Lite	0.929	0.776	0.003	146.80
GPT-3.5-Turbo	0.890	0.533	0.015	208.66
DeepSeek-V3	0.986	0.699	0.003	231.11
Qwen3.5-27B*	0.971	0.852	0.119	1395.6

* Qwen3.5-27B with deep thinking mode

TABLE IX
MANUAL EFFORT FOR EXTENDING MORPH TO TARGET DBMSS.

Target DBMS	Auto Rules	Manual	Ratio
DuckDB	534	42	7.3%
ClickHouse	1,362	46	3.3%
MonetDB	398	26	6.1%
Virtuoso	782	22	2.7%
Total	3,076	136	4.2%

and each correction typically takes 1–3 minutes for an engineer with DBMS expertise.

VI. RELATED WORK

A. DBMS Fuzzing

Fuzzing is one of the most effective automated testing techniques for identifying DBMS vulnerabilities. Generation-based tools construct queries via predefined rules or semantic oracles. SQLsmith [9] randomly instantiates queries from a grammar. SQLancer [34], [38] applies oracles to detect logic bugs, and SQLRight [35] combines coverage-guided mutation with logic-bug oracles. sqllogictest [39] validates query results across database engines with a shared test format. DynSQL [40] leverages dynamic feedback to guide query mutation. More recently, clause-guided and differential methods have further improved logic-bug detection [41]–[45]. However, predefined templates can limit complex scenarios [46]. Mutation-based fuzzers generate test cases by continuously evolving a pool of seed inputs, rather than constructing them from scratch. SQUIRREL [12] uses IR-based mutations, GRIFFIN [10] uses grammar-free metadata replacement, and LEGO [11] combines SQL statement sequences. These fuzzers are effective once suitable seeds or generation rules are available, but they do not directly address how to obtain high-quality seeds for emerging DBMSs. Seed-selection and scheduling techniques, such as corpus minimization [47] and power schedules [48], choose or prioritize seeds within an existing corpus, whereas MORPH constructs the initial corpus for target DBMSs that lack one; the two directions are orthogonal and can be combined.

Unlike these works, MORPH focuses on the upstream *seed acquisition* problem by transferring mature DBMS test cases into target-compatible seed sequences.

B. LLM-Based Seed Generation

Recent work has explored LLMs for generating or enriching testing inputs. Fuzz4All [49] uses LLMs as a general input generator, and ShQveL [50] synthesizes SQL fragments for database testing. SmartFuzz [37] uses LLMs to generate new DBMS-specific seeds from features extracted from official DBMS documentation and historical bug-triggering inputs. In contrast, MORPH transfers existing test cases across DBMSs and uses cross-statement dependencies and cumulative schema and type context for translation and repair. The two approaches construct seeds from different sources and can be combined to construct initial seeds for DBMS fuzzing. SEDAR [17] is the closest work to MORPH because it also transfers existing DBMS test cases into fuzzing seeds; its compatibility mechanism is described in Section II.

Unlike SEDAR, which translates each statement of a test case individually under its referenced source-side sub-schema, MORPH performs AQT-guided instantiation that maintains target-side context across statements, where explicit dependencies, nested structures, and clause-level relationships support local repair with parser and execution feedback before mutation.

C. Cross-Dialect SQL Translation

General-purpose SQL dialect translators, such as SQLGlot [15] and jOOQ [16], exploit the deterministic structure of SQL dialects and encode dialect mappings as parser or transpiler rules, making them useful for recurring statement-level conversions in query migration and application compatibility. However, constructing and maintaining these rules requires significant dialect expertise. Recent studies have reduced this manual effort by learning or deriving translation behaviors automatically. CrackSQL [23] combines functionality-based query processing, cross-dialect syntax embedding with contrastive learning, and local-to-global translation. RISE [22] reduces a complex query or procedural artifact to a simpler form, learns the local dialect rewrite from that reduced form, and applies the rewrite back to the original artifact. While this strategy improves robustness, RISE processes one self-contained query or procedural artifact at a time, rather than an ordered test sequence of independently executed statements in which later statements may depend on the state produced by earlier ones. These systems target query migration. In contrast, QTRAN [24] targets oracle transfer for logic-bug detection: it uses dialect knowledge and retrieval to translate a query and regenerate its target-side mutant so that the metamorphic relation is preserved.

MORPH adopts related techniques, including structural decomposition, localized translation, and dialect-aware retrieval, but targets fuzzing seed transfer. It focuses on maintaining evolving target-side context across an ordered test sequence and on producing seeds that remain parsable and mutable by downstream fuzzers. To this end, MORPH transfers the entire seed sequence as normalized per-statement forms within an AQT while preserving cross-statement dependencies and evolving schema context.

VII. CONCLUSION

In this paper, we propose MORPH to construct fuzzing seeds with structure-aware cross-dialect SQL transfer. It extracts dialect-agnostic AQTs, instantiates them through Dialect-Aware AQT Instantiation, and applies adaptive sanitization for cross-dialect DBMS fuzzing. Compared with the state-of-the-art approach SEDAR, MORPH covers 14% more branches and finds 16 more bugs in 24 hours. Across four popular DBMSs, MORPH uncovers 28 previously unknown bugs, including 20 confirmed and 14 already fixed.

VIII. DATA AVAILABILITY

All artifacts and data used are publicly available in our anonymous repository [51].

REFERENCES

[1] T. P. G. D. Group, "pgsql-bugs," https://www.postgresql.org/list/pgsql-bugs/, 2025, accessed: November 11, 2026.

[2] MySQL, "Mysql bug home," https://bugs.mysql.com/, 2025, accessed: November 11, 2026.

[3] M. Zalewski, "american fuzzy lop," http://lcamtuf.coredump.cx/afl/, accessed: November 11, 2026.

[4] "Libfuzzer," https://www.llvm.org/docs/LibFuzzer.html, accessed: November 11, 2026.

[5] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "Afl++ : Combining incremental steps of fuzzing research," in *USENIX Workshop on Offensive Technologies (WOOT)*, 2020.

[6] J. Liang, Y. Jiang, Y. Chen, M. Wang, C. Zhou, and J. Sun, "PAFL: Extend fuzzing optimizations of single mode to industrial parallel mode," in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2018, pp. 809–814.

[7] "Sqlsmith score list," https://github.com/anse1/sqlsmith/wiki#score-list, accessed: November 11, 2026.

[8] "Bugs found in database management systems," https://www.manuelrigger.at/dbms-bugs, accessed: November 11, 2026.

[9] SQLsmith Contributors, "Sqlsmith," 2015, accessed: 2026-01-26. [Online]. Available: https://github.com/anse1/sqlsmith

[10] J. Fu, J. Liang, Z. Wu, M. Wang, and Y. Jiang, "Griffin: Grammar-free dbms fuzzing," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.

[11] J. Liang, Y. Chen, Z. Wu, J. Fu, M. Wang, Y. Jiang, X. Huang, T. Chen, J. Wang, and J. Li, "Sequence-oriented dbms fuzzing," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 668–681.

[12] R. Zhong, Y. Chen, H. Hu, H. Zhang, W. Lee, and D. Wu, "Squirrel: Testing database management systems with language validity and coverage feedback," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 955–970.

[13] "Postgresql github," https://github.com/postgres/postgres, accessed: November 11, 2026.

[14] "Duckdb github," https://github.com/duckdb/duckdb, accessed: November 11, 2026.

[15] SQLGlot Contributors, "SQLGlot: Python sql parser and transpiler," 2026, accessed: 2026-06-20. [Online]. Available: https://github.com/tobymao/sqlglot

[16] jOOQ Contributors, "jOOQ: Type safe sql in java," 2026, accessed: 2026-06-20. [Online]. Available: https://www.jooq.org/

[17] J. Fu, J. Liang, Z. Wu, and Y. Jiang, "Sedar: Obtaining high-quality seeds for dbms fuzzing via cross-dbms sql transfer," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.

[18] "MonetDB NextRelease milestone," https://github.com/MonetDB/MonetDB/milestone/1?closed=1, 2025, accessed: 2026-03-25.

[19] M. Wang, Z. Wu, X. Xu, J. Liang, C. Zhou, H. Zhang, and Y. Jiang, "Industry practice of coverage-guided enterprise-level dbms fuzzing," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2021, pp. 328–337.

[20] SQLGlot Contributors, "SQLGlot api documentation: parse and transpile," 2026, accessed: 2026-10-24. [Online]. Available: https://sqlglot.com/sqlglot.html

[21] jOOQ Contributors, "jOOQ user manual: Sql interpreter," 2026, accessed: 2026-10-24. [Online]. Available: https://www.jooq.org/doc/latest/manual/sql-building/sql-interpreter/

[22] X. Xie, Y. Zhang, W. Dou, Y. Gao, Z. Cui, J. Song, R. Yang, and J. Wei, "Rise: Rule-driven sql dialect translation via query reduction," *arXiv preprint arXiv:2601.05579*, 2026.

[23] W. Zhou, Y. Gao, X. Zhou, and G. Li, "Cracking sql barriers: An llm-based dialect translation system," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–26, 2025.

[24] L. Lin, Q. Zhu, H. Chen, Z. Wang, R. Wu, and X. Xie, "Qtran: Extending metamorphic-oracle based logical bug detection techniques for multiple-dbms dialect support," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 731–752, 2025.

[25] X. Zhou, C. Chai, G. Li, and J. Sun, "Database meets artificial intelligence: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 3, pp. 1096–1116, 2020.

[26] Google, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," 2025. [Online]. Available: https://arxiv.org/abs/2507.06261

[27] "Db-engines ranking — popularity ranking of database management systems," https://db-engines.com/en/ranking, accessed: November 11, 2026.

[28] "The database system to speed up your analytical jobs," https://www.monetdb.org/, 1 2024, accessed: November 11, 2026.

[29] "Virtuoso github." https://github.com/openlink/virtuoso-opensource, accessed: November 11, 2026.

[30] "Query billions of rows in milliseconds," https://clickhouse.com/, 1 2024, accessed: November 11, 2026.

[31] S. Bhosale, M. T. Patil, and M. P. Patil, "Sqlite: Light database system," *Int. J. Comput. Sci. Mob. Comput*, vol. 44, no. 4, pp. 882–885, 2015.

[32] "Mysql," https://www.mysql.com/, 1 2024, accessed: November 11, 2026.

[33] "Postgresql," https://www.postgresql.org/, 1 2024, accessed: November 11, 2026.

[34] M. Rigger and Z. Su, "Sqlancer," 2023, accessed: 2026-01-26. [Online]. Available: https://github.com/sqlancer/sqlancer

[35] Y. Liang, S. Liu, and H. Hu, "Detecting Logical Bugs of DBMS with Coverage-based Guidance," in *Proceedings of the 31st USENIX Security Symposium (USENIX 2022)*, Boston, MA, aug 2022.

[36] C. chen, "Squirrel website," https://github.com/s3team/Squirrel, accessed: November 11, 2026.

[37] L. Lin, J. Hong, Y. Zhuang, and R. Wu, "SmartFuzz: Leveraging large language models and feature composition to generate high-quality seeds for database fuzzing," *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA2, pp. 823–851, 2026.

[38] M. Rigger and Z. Su, "Testing database engines via pivoted query synthesis," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 667–682.

[39] SQLite, "sqllogictest," https://www.sqlite.org/sqllogictest/, 2008.

[40] Z.-M. Jiang, J.-J. Bai, and Z. Su, "{DynSQL}: Stateful fuzzing for database management systems with complex and valid {SQL} query generation," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4949–4965.

[41] J. Wei, P. Chen, K. Lu, J. Dai, and X. Sun, "Sqlaser: Detecting dbms logic bugs with clause-guided fuzzing," *arXiv preprint arXiv:2407.04294*, 2024.

[42] J. Ba and M. Rigger, "Testing database engines via query plan guidance," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2060–2071.

[43] J. Song, W. Dou, Z. Cui, Q. Dai, W. Wang, J. Wei, H. Zhong, and T. Huang, "Testing database systems via differential query execution," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2072–2084.

[44] Z. Cui, W. Dou, Q. Dai, J. Song, W. Wang, J. Wei, and D. Ye, "Differentially testing database transactions for fun and profit," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.

[45] Z. Hao, Q. Huang, C. Wang, J. Wang, Y. Zhang, R. Wu, and C. Zhang, "Pinolo: Detecting logical bugs in database management systems with approximate query synthesis," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 345–358.

[46] X. Gao, Z. Liu, J. Cui, H. Li, H. Zhang, K. Wei, and K. Zhao, "A comprehensive survey on database management system fuzzing: Techniques, taxonomy and experimental comparison," *arXiv preprint arXiv:2311.06728*, 2023.

[47] A. Herrera, H. Gunadi, S. Magrath, M. Norrish, M. Payer, and A. L. Hosking, "Seed selection for successful fuzzing," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 230–243.

[48] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as markov chain," in *IEEE Transactions on Software Engineering*, 2017.

[49] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[50] S. Zhong and M. Rigger, "Testing database systems with large language model synthesized fragments," *arXiv preprint arXiv:2505.02012*, 2025.

[51] "Morph artifact and experimental data," Anonymous repository, 2026. [Online]. Available: https://anonymous.4open.science/r/MORPHArtifact-8E44/